ARTIFICAL INTELLIGENCE - MODULE I

SYLLABUS – Software agents, Agent Charcteristics, Agent Topology, Agent Oriented Programming, Java Implementation of Intelligent Agents.

Q) Explain Software Agents/ Intelligent agents with example and figure – explain software agents, fig below, agent as ascription and description and some applications, examples , features based on marks

SOFTWARE AGENTS

A **software Agent** is an autonomous computer program that acts on behalf of a user or other program.

- It provides Software abstraction
- Its autonomous and standalone
- Can communicate with users and other agents
- It can respond to change.

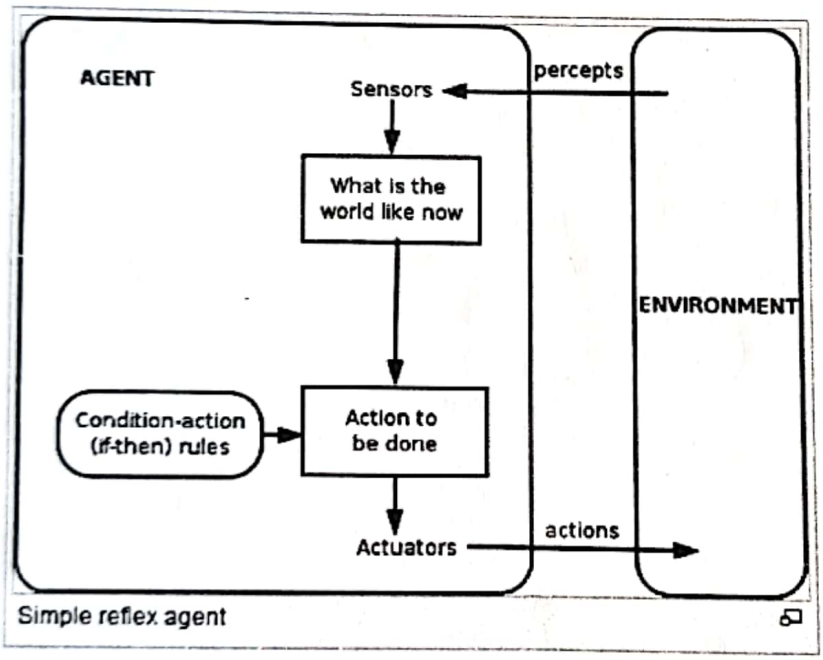


Figure explanation – An agent takes input from environment using sensors like temperature pressure etc, Checks for conditions using If-then rules like is temp above a certain level ...If needed appropriate action is taken using actuators etc, which is reflected back in the environment,

Scanned with CamScanner

'Agent' as an Ascription

We expect a software agent to act on behalf of someone to carry out a particular task which has been delegated to it. But since it is tedious to have to spell out every detail, we would like our agents to be able to infer what we mean from what we tell it. Agents can only do this if they "know" something about the context of the request. The best agents, then, would not only need to exercise a particular form of expertise, but also take into account the peculiarities of the user and situation.

Agent' As a Description

A software entity which functions continuously and autonomously in a particular environment, is often inhabited by other agents and processes. The requirement for continuity and autonomy derives from our desire that an agent be able to carry out activities in a flexible and intelligent manner that is responsive to changes in the environment without requiring constant human guidance or intervention. Ideally, ***an agent that functions continuously in an environment over a long period of time would be able to learn from its experience***. In addition, we expect an agent that inhabits an environment with other agents and processes to be able to communicate and cooperate with them, and perhaps move from place to place in doing so.

Q) Example scenarios where Software agents are used

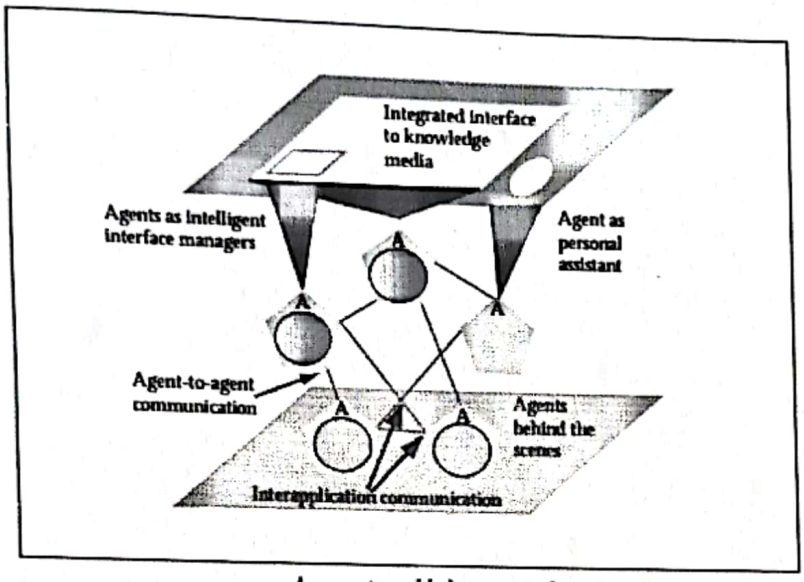
Examples of Software agents

1. **Buyer Agents** - for shopping - Buyer agents travel around a network (e.g. the internet) retrieving information about goods and services.
2. **User Agents**
   - Checking / sorting mails based on priority
   - Play computer games as opponents
   - Customize news and Ads based on preference
   - Automatic filling f web forms
3. **Data Mining Agents** - This agent uses information technology to find trends and patterns in an abundance of information from many different sources. The user can sort through this information in order to find whatever information they are seeking.

4. **Monitoring and Surveillance Agents** - are used to observe and report on equipment, usually computer systems. The agents may keep track of company inventory levels, observe competitors' prices and relay them back to the company, watch stock manipulation by insider trading and rumors, etc.

Scanned with CamScanner

Architecture of Agent enabled system architecture



An agent-enabled system architecture.

Q) Different Types of Software Agents, can also be asked to distinguish between the different agents.

Intelligent agents - in particular exhibiting some aspect of artificial intelligence, such as learning and reasoning

Autonomous agents - capable of modifying the way in which they achieve their objectives

***Distributed* agents** - being executed on physically distinct computers),

Multi-agent systems distributed agents that do not have the capabilities to achieve an objective alone and thus must communicate

Mobile agents - is a composition of computer software and data which is able to migrate from one computer to another autonomously and continue its execution on the destination computer.

Scanned with CamScanner

Q) What are the applications / functions /Usage / Need for Software agents

1. **Simplifying the complexities of Distributed Computing –**

   - **To overcome Barriers to Intelligent Interoperability** to exchange and make use of information.
   - **Incorporating Agents as Resource Managers** - A higher level of interoperability would require knowledge of the capabilities of each system, so that secure task planning, resource allocation, execution, monitoring, and, possibly, intervention between the systems could take place. To accomplish this, an intelligent agent could function as a global resource manager

2. **Overcoming limitations of User Interface Problems like -**

   - **Scalability**: Agents can be equipped with search and filtering capabilities that run in the background to help people explore vast sources of information.
   - **Scheduled or event-driven actions**: Agents can be instructed to execute tasks at specific times or automatically "wake up" and react in response to system-generated events.
   - **Abstraction and delegation**: Agents can be made extensible and composable in ways that common iconic interface objects cannot. Because we can "communicate" with them, they can share our goals, rather than simply process our commands. They can show us how to do things and tell us what went wrong
   - **Flexibility and opportunism**: Because they can be instructed at the level of goals and strategies, agents can find ways to "work around" unforeseen problems and exploit new opportunities as they help solve problems.
   - **Task orientation**: Agents can be designed to take the context of the person's tasks and situation into account as they present information and take action.
   - **Adaptivity:** Agents can use learning algorithms to continually improve their behavior by noticing recurrent patterns of actions and events

3. **Other functions**

   - Data collection and filtering
   - Pattern recognition
   - Event Notification
   - Data Presentation
   - Planning and Optimization

Scanned with CamScanner

- Rapid Response implementation

Q) List and explain the Characteristics / Attributes / Features of software Agents
(can explain about software agents, functions examples in addition based upon marks, but the below points should be there in answer.)

Characteristics of Agents

1. **Reactivity**: the ability to selectively sense and act
2. **Autonomy**: goal-directedness, proactive and self-starting behavior
3. **Collaborative behavior**: can work in concert with other agents to achieve a common goal
4. **"Knowledge-level"** - communication ability: the ability to communicate with persons and other agents with language more resembling humanlike "speech acts" than typical symbol-level program-to-program protocols
5. **Inferential capability**: can act on abstract task specification using prior knowledge of general goals and preferred methods to achieve flexibility; goes beyond the information given, and may have explicit models of self, user, situation, and/or other agents.
6. **Temporal continuity**: persistence of identity and state over long periods of time
7. **Personality:** the capability of manifesting the attributes of a "believable" character such as emotion
8. **Adaptivity:** being able to learn and improve with experience
9. **Mobility**: being able to migrate in a self-directed way from one host platform to another

Q) Classification/Characterization/Taxonomies of Software Agents

(each classification can be asked separately too like based on degree of problem solving, based on 3 dimensions of agency,intelligence and mobility ...etc....)

1. **weak and strong notions of agency:** agents of the latter variety are designed to possess explicit mentalistic or emotional qualities
2. **agents by degree of problem-solving capability:**
   - A **reactive agent** reacts to changes in its environment or to messages from other agents
   - An **intentional agent** is able to reason on its intentions and beliefs, to create plans of actions, and to execute those plan
   - **social agent** possesses explicit models of other agents.

3. **Characterization based on three dimensions - agency, intelligence, and mobility**

Scanned with CamScanner

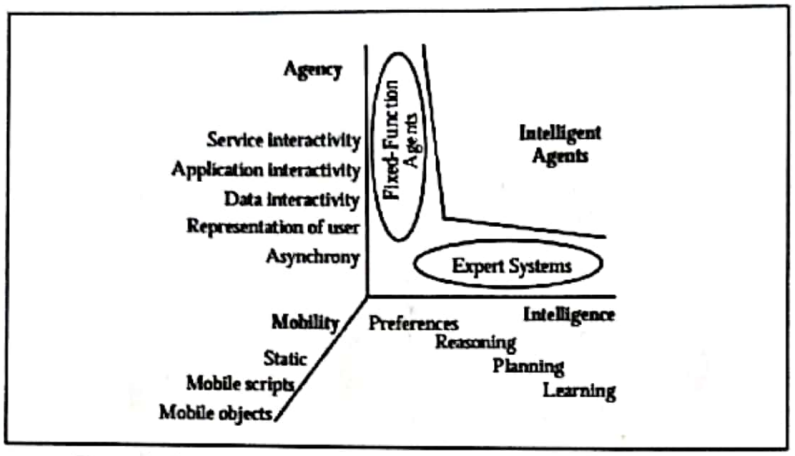


Figure 1. Scope of intelligent agents (Adapted from Gilbert et al. 1995).

"**Agency** is the degree of autonomy and authority vested in the agent, and can be measured at least qualitatively by the nature of the interaction between the agent and other entities in the system. At a minimum, an agent must run asynchronously. The degree of agency is enhanced if an agent represents a user in some way... A more advanced agent can interact with data, applications, services[or] other agents.

Intelligence is the degree of reasoning and learned behavior: the agent's ability to accept the user's statement of goals and carry out the task delegated to it. At a minimum, there can be some statement of preferences... Higher levels of intelligence include a user model... and reasoning.... Further out on the intelligence scale are systems that learn and adapt to their environment, both in terms of the user's objectives, and in terms of the resources available to the agent

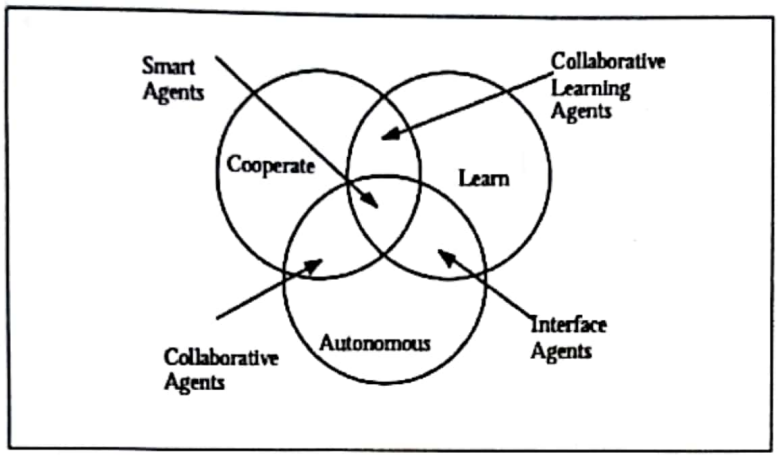
Mobility is the degree to which agents themselves travel through the network... Mobile scripts may be composed on one machine and shipped to another for execution... [Mobile objects are] transported from machine to machine in the middle of execution, and carrying accumulated state data with them."

4. **Based on Mobility - as static or mobile**

5. **Presence of a symbolic reasoning model - as deliberative or reactive.**

6. Based on **Exhibition of ideal and primary attributes** - such as **autonomy, cooperation, learning.** From these characteristics, Nwana derives four agent types: **collaborative, collaborative learning, interface, and smart**

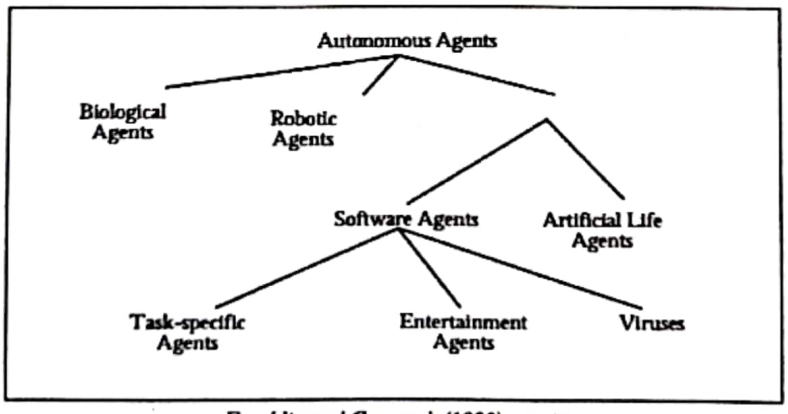
Scanned with CamScanner



Typology based on Nwana's (Nwana 1996) primary attribute dimension.

7. Based on **Roles, as information or Internet**

8. **Based on Hybrid philosophies**, which combine two or more approaches in a single agent
9. **Based on Secondary attributes**, such as versatility, benevolence, veracity, trustworthiness, temporal continuity, ability to fail gracefully, and mentalistic and emotional qualities.
10. After developing this typology, Nwana goes on to describe ongoing research in seven categories:
    - **collaborative agents**
    - **interface agents**
    - **mobile agents**
    - **information/Internet agents**
    - **reactive agents**
    - **hybrid agents**
    - **smart agents**



Franklin and Graesser's (1996) agent taxonomy.

Scanned with CamScanner

Agent - Oriented Programming (AOP)

→ OOP proposes viewing a computational system as made up of modules that are able to communicate with one another and that have individual ways of handling incoming messages

→ AOP specializes the framework by fixing the state (mental state) of the modules (agents) to consist of precisely defined components called beliefs (beliefs about the resulting systems about the world, about themselves and about one another), capabilities choices and possibly other similar notations

→ A computation consists of these agents' informing, requesting, offering, accepting, rejecting, competing and assisting one another.

Scanned with CamScanner

Q) Explain Agent Oriented Programming (explain framework components, compare with OOP, explain and draw figure of generic agent interpreter)

Agent-oriented programming can be thought of as a specialization of object-oriented programming approach, with constraints on what kinds of state-defining parameters, message types, and methods are appropriate. From this perspective, an agent is essentially "an object with an attitude." An agent's "mental state" consists of components such as beliefs, decisions, capabilities, and obligations. Agent programs control the behavior and mental state of agents. These programs are executed by an agent interpreter. In the spirit of speech act theory, interagent communication is implemented as speech act primitives of various types, such as inform, request, or refrain. An agent interpreter assures that each agent will iterate through two steps at regular intervals:

1) read the current messages and update its mental state (including beliefs and commitments), and

2) execute the commitments for the current time, possibly resulting in further belief change.

Agent Orient Programming (AOP) Framework

A complete AOP system include 3 primary components

1. **A restricted formal language** with clear syntax and semantics for describing mental state. The mental state will be defined uniquely by several modalities, such as belief and commitment.

2. **An Interpreter programming language** in which to program agents with primitive commands such REQUEST and INFORM. The semantics of programming language will depend in part on the semantics of mental state.

3. **An "agentifier"** converting neural devices into programmable agents.

Comparison between Object Oriented Programming and Agent Oriented Programming

Scanned with CamScanner

OOP versus AOP

	OOP	AOP
Basic unit	object	agent
Parameters defining state of basic unit	unconstrained	beliefs, commitments, capabilities, choices, ...
Process of computation	message passing and response methods	message passing and response methods
Types of message	unconstrained	inform, request, offer, promise, decline, ...
Constraints on methods	none	honesty, consistency, ...

The behaviour of agent is governed by programs; each agent is controlled by its own private program.

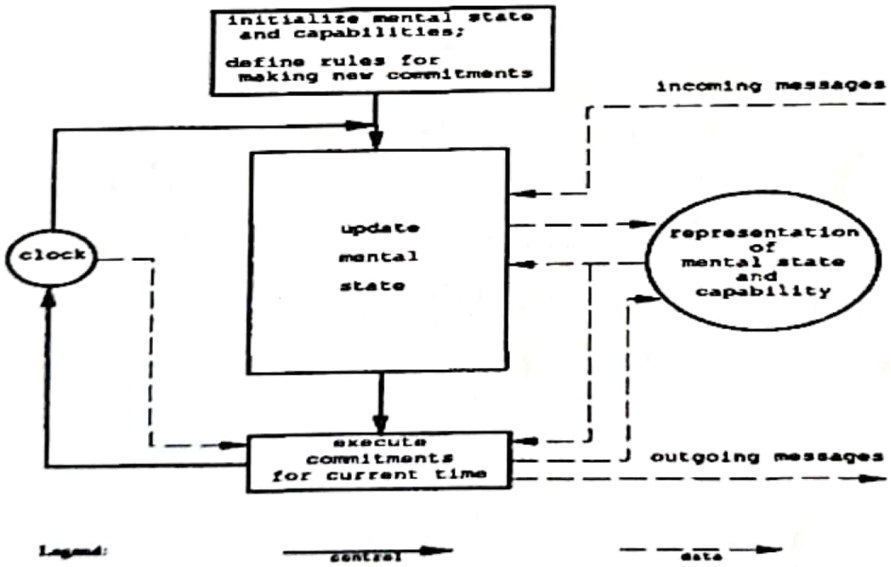
Generic Agent Interpreter

The basic loop

The behavior of agents is, in principle, quite simple. Each agent iterates the following two steps at regular intervals:

(1) read the current messages, and update your mental state, including your beliefs and commitments (the agent program is crucial for this update);

(2) execute the commitments for the current time, possibly resulting in further belief change (this phase is independent of the agent's program).



Flow diagram of a generic agent interpreter.

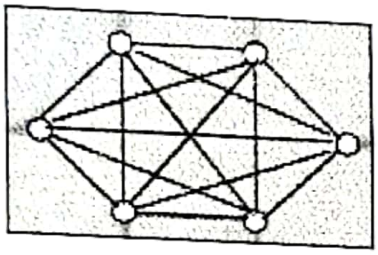
Q) Illustrate/explain the various AGENT TOPOLOGY (draw and briefly explain the different network connections also draw typology based on Nwana figure in next page

Agent programs themself are not logical entities, but their control and data structures refer to the mental state of the agent using the logical language.

Scanned with CamScanner

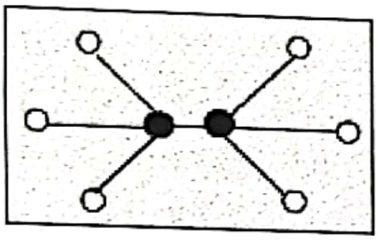
along with the 4 network topologies below, can include other topologies you studied in networks)

Agents in a multi-agent system (MAS) could interact and cooperate in many different ways. The topology of agent interaction determines how the agents control and communicate with each other, what are the control and communication capabilities of each agent and the whole system, and how efficient the control and communications are. In consequence, the topology affects the agents' ability to share knowledge, integrate knowledge, and make efficient use of knowledge in MAS.There are four major MAS topologic models.



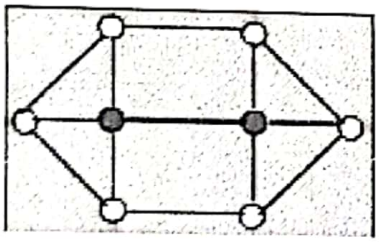
- uniform interconnection of agents in a cooperative environment.
- every agent node can directly interact with all other node
- same internal structure, operational goal domain knowledge

Fig. 1. Web-like topology of agent interaction.



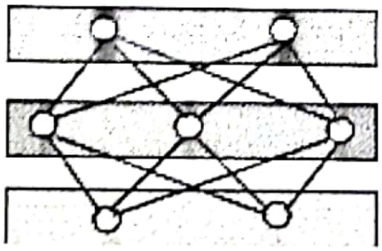
- Supervisory (facilitator) agent – coordinated or administered the activity of agent.
- agents are more under control and stipulation

Fig. 2. Star-like topology of agent cooperation.



- Each agents cooperates with a group (an agency) of agents in its neighborhood (in terms of functional connections) that is a subset of agents in its neighborhood group.
- each group has a administrator

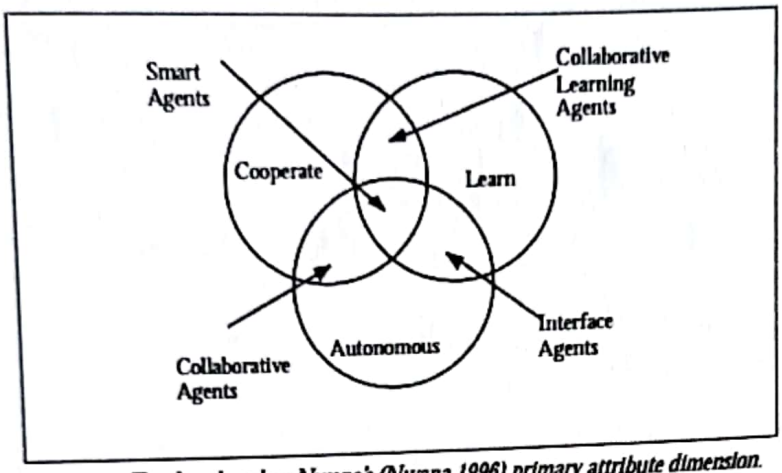
Fig. 3. Grid-like topology of agent cooperation. interaction to agent not in the neighbourhood may pass through the facilitators of the neighbourhood



- Agents are grouped in layers
- layers a organized in hierarchy
- agents in each layers are not connected
- agents b/w layers are fully connected
- control and coordinate of the agent at each layer are through the agents at high level

Scanned with CamScanner

Fig. 4. Hierarchical collective topology of agent cooperation.



Typology based on Nwana's (Nwana 1996) primary attribute dimension.

Q) Intelligent agent implementation / Intelligent agent implementation in JAVA. - JADE(..full till the end, each and every point has to be written with figures and examples)

Intelligent Agent Implementation in JAVA

Intelligent agents

Intelligent Agents are software components which schedule and complete tasks on a user's behalf autonomously or semi-autonomously. As with most systems, the level of autonomous processing will vary depending on the application domain and the user.

The Intelligent Agent system characteristics include the following:

Ø **Intelligence**: the agent automatically customizes its self to users' requests based on previous experience. The agent also automatically adapts to changes in its environment.

Ø **Autonomy**: an agent is able to take the initiative and exercise a nontrivial degree of control over its own actions through the implemented roles.

Ø **Cooperation**: An agent does not blindly obey commands, but makes suggestions to modify requests or ask clarification questions. It also cooperates with other agents to manage the project. In order to communicate with one another, agents use language, protocol and ontology for exchanging information, knowledge, and sharing common understanding of the structure of information.

Scanned with CamScanner

FIPA Foundation for Intelligent Physical Agents

– body for developing and setting computer software standards for heterogeneous and interacting agents and

Ø **Mobility**: A single agent can be deployed in different process with different interfaces. Agent based s/ms

There are two main configurations of agent based systems:

- single agent systems (one agent carrying out tasks) or
- multi-agent systems (numerous agents co-operating to complete tasks).

JADE Java Agent DEvelopment Framework

JADE is a software framework and middle-ware enabling technology for developing multi-agent systems. As a FIPA-compliant agent platform, JADE provides an implementation for the following components:

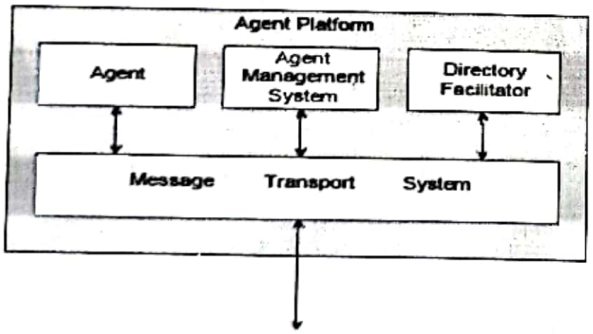
Java

Ø **Agent Management System (AMS)**. This agent is responsible for controlling access to the platform, authentication and registration of participating agents.

Ø **Directory Facilitator (DF)**. This agent provides a yellow page service to the agents in the platform.

Ø **Agent Communication Channel (ACC)**. This agent provides a white page service. It also supports inter-agent communication and inter-operability within and across different platforms.

JADE also provides an implementation of the full FIPA communication model, i.e., agent interaction protocols (AIPs), FIPA Agent Communication Language (ACL), SL content language, ontology support and, transport protocols.

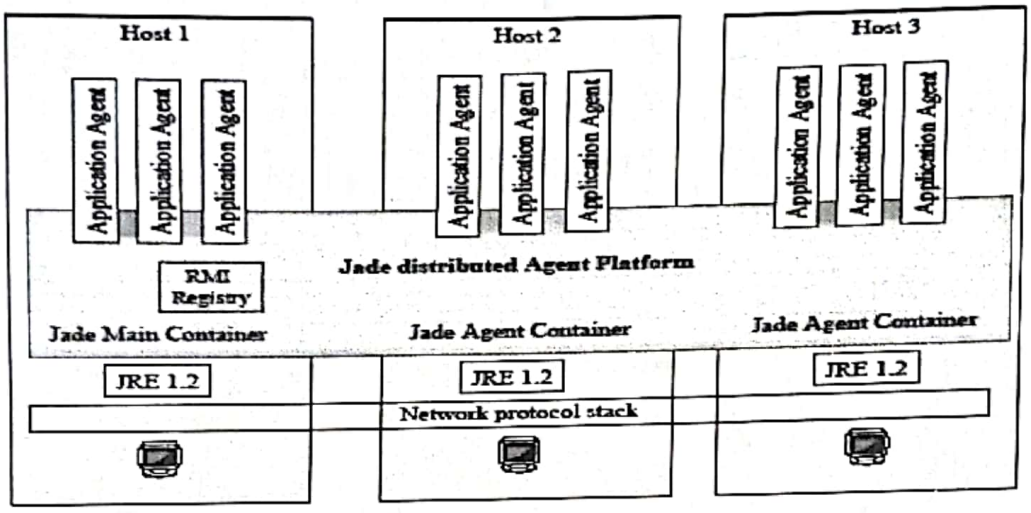


(Reference architecture of a FIPA Agent Platform)

JADE is written in Java language and is made of various Java packages, giving application programmers both ready-made pieces of functionality and abstract

Scanned with CamScanner

interfaces for custom, application dependent tasks. Java was the programming language of choice because of its many attractive features, particularly geared towards object-oriented programming in distributed heterogeneous environments; some of these features are Object Serialization, Reflection API and Remote Method Invocation (RMI).



(JADE Agent Platform distributed over several containers)

JADE - Java Agent DEvelopment Framework is a framework to develop multi-agent systems.JADE is a middleware that facilitates the development of multi-agent systems

JADE includes

- A runtime environment where JADE agents can "live" and that must be active on a given host before one or more agents can be executed on that host.

- A library of classes that programmers have to/can use (directly or by specializing them) to develop their agents.

Scanned with CamScanner

A suite of graphical tools that allows administrating and monitoring the activity of running agents

Each running instance of the JADE runtime environment is called a **Container** as it can contain several agents.

The set of active containers is called a **Platform.**

- A single special Main container must always be active in a platform and all other containers register with it as soon as they start.

- If another main container is started somewhere in the network it constitutes a different platform

- The following command line launches a Main Container activating the JADE management GUI (-gui) option

- <classpath> must include all jade classes plus all required application-specific classes.

```
java -cp <classpath> jade.Boot –gui
```

AMS

- A main container differs from normal containers as it holds two special agents (automatically started when the main container is launched).
- The AMS (Agent Management System) provides the naming service and represents the authority in the platform (for instance it is possible to create/kill agents on remote containers by requesting that to the AMS).

DF

- The DF (Directory Facilitator) provides a Yellow Pages service where an agent can find other agents providing the services he requires in order to achieve his goals.

BOOK TRADING EXAMPLE (for Intelligent agent implementation using JAVA)

- some agents sell books and other agents buy books on behalf of their users.
- Each buyer agent receives the title of the book to buy as a command line argument
- Periodically requests all known seller agents to provide an offer.
- As soon as an offer is received the buyer agent accepts it and issues a purchase order.

Scanned with CamScanner

- If more than one seller agent provides an offer the buyer agent accepts the best one.
- The buyer agent terminates.
- When asked to provide an offer for a book they check if the requested book is in their catalogue and in this case reply with the price
- Otherwise they refuse. When they receive a purchase order they serve it and remove the requested book from their catalogue.
- Creating a JADE agent is as simple as defining a class extending the jade.core.Agent class and implementing the setup() method as shown in the code below.

```
import jade.core.Agent;

public class BookBuyerAgent extends Agent {

protected void setup() {

// Printout a welcome message

System.out.println("Hello! Buyer-agent "+getAID().getName()+" is ready.");

}}
```

The setup() method is intended to include agent initializations

Scanned with CamScanner